Consensus Guide

Ledger state, block validation, and the meaning of confirmation

m@rek.onl

Abstract

This volume of *The Zcash Arboretum* explains how computers agree on an ordered record of payments, check proposed additions, and account for the money those additions create or transfer. It assumes the *Math Guide*, especially "Language, integers, and encodings" and "Probability and computation", and the *Crypto Guide*, "Hash functions and key derivation" and "Authenticated trees and append-only histories". The payment's private computations enter through explicit contracts specifying what their evidence must establish. The exposition does not implement the internals of every historical transaction format. It is non-normative: the protocol specification and the applicable consensus ZIPs determine the rules.

Contents

1 Ledger state and transaction acceptance

A payment changes who can spend value. It does not move a physical object between machines. The *ledger* is the accepted record of such changes. A *transaction* is one proposed change, with the data needed to check it. A *node* is a participating computer; a validating node checks the rules itself. Each validating node maintains a public state, receives a proposed change, and checks that the change follows the rules. Privacy changes the evidence used in that check; it does not remove the need for a public decision about whether a particular change has already occurred.

A *transparent output* publicly records an amount and conditions for spending it. A *shielded note* instead describes an amount and receiving capability privately; the chain records a commitment to that description. A *shielded pool* is a separately accounted collection of such notes. Commitment hiding and binding have their security definitions in the *Crypto Guide*, "Groups, commitments, and Sinsemilla"; this volume uses the interface without assuming any one payment construction.

One *zatoshi* is the indivisible accounting unit, and 10^8 zatoshis equal one ZEC. All consensus amounts are integers. A negative public balance will mean a transfer *into* a shielded pool, not a negative-valued private note. The distinction between integer arithmetic and arithmetic modulo a prime is developed in the *Math Guide*, "Prime fields and modular arithmetic".

1.1 State belongs to a branch

A *block* contains an ordered list of transactions and a *header*, a fixed-format summary naming its parent block by hash. The *genesis block* is the prescribed starting block. Starting there, these parent links form a tree. A *branch* is one path through that tree. Its height counts edges from genesis; competing blocks can therefore have the same height without being the same block.

An *anchor* is a previously accepted note-tree root against which a private existence proof is checked. A *nullifier* is a deterministic public spend tag whose repetition is forbidden within its pool. The note tree is an append-only tree of note commitments; the generic tree and its root are constructed in the *Crypto Guide*, "Authenticated trees and append-only histories".

For a branch tip b, write its state schematically as

$$S_b = (h, U, \{T_P, A_P, N_P, B_P\}_{P \in \mathcal{P}}, D, \mathcal{H}).$$

Here h is the height; U is the map of unspent transparent outputs; $\mathcal{P}$ is the set of shielded pools recognised by the active rules; T_P is pool P's append-only note-commitment tree; A_P records roots eligible as anchors; N_P is its set of revealed nullifiers; and B_P is its public chain value pool balance. The *deferred development pool* is publicly tracked value set aside under prescribed allocation rules; its balance is D. The authenticated block-history state is $\mathcal{H}$.

The state stores commitments and spend tags, not private note amounts or owners. The commitment and nullifier mechanisms must be provided by the payment protocol. The ledger needs their interfaces: append a commitment, recognise an eligible root, and reject a repeated tag.

A state indexed only by height is ambiguous. If blocks b and b' have the same height, their output maps, nullifier sets and note trees can differ. Every state-dependent query must identify a branch, either explicitly by hash or implicitly by a stable branch snapshot.

1.2 A typed payment-validation contract

An *ordinary transaction* transfers existing value; it is not the special transaction permitted to issue the block's subsidy, or newly created money. It supplies transparent input references and outputs, zero or more shielded components, and *authorising data*: proofs and signatures used to validate the proposed effects. A proof's private input is its *witness*; its public inputs are visible to the verifier. Their security contracts and signatures are introduced in the *Crypto Guide*, "Knowledge, simulation, and Schnorr" and "RedPallas signatures". For each shielded component its public interface contains:

- an ordered commitment list $C_P(t)$ to append;

- a nullifier list $N_P(t)$ to insert;
- any required anchor or anchors;
- a signed public balance $b_P(t)$, positive for net value leaving the pool;
- a proof and signatures establishing the protocol-specific relationship between these public objects and private witnesses.

A list, unlike a set, retains order and multiplicity. Checking only that $N_P(t) \cap N_P = \varnothing$ is insufficient: the list itself must not contain a repeated nullifier.

The payment mechanism must establish that nonzero private inputs existed under the claimed anchors, that their spend tags were computed correctly, that the spender authorised this transaction, and that private input value minus private output value equals $b_P(t)$. A zero-knowledge proof alone need not establish all these claims. A design may divide them between a proof, signatures approving spends and a separate signature enforcing the combined value balance. The ledger invokes the complete specified verifier, not whichever subset is easiest to batch.

The network parameters specify which rules apply at each height. A format's *version* and *version-group identifier* select its byte layout and interpretation. Its *consensus branch identifier* separates cryptographic computations belonging to different upgrade rules; it does not name a block-tree branch. A *verification key* is the public circuit-specific input to proof verification, not a spending key. Field and point encodings use the *Math Guide*, "Prime fields and modular arithmetic" and "Pasta curves and their encodings". An input is *mature* when its required waiting interval has elapsed. A transaction's lock time limits when it can begin to be accepted; its expiry condition limits how late it can be accepted.

Definition 1.1 (Ordinary transaction acceptance)**.** For network parameters θ, candidate block height h and a branch-local state S, transaction acceptance is the conjunction of:

1. *Well-formedness:* canonical parsing, bounded lengths, valid field and point encodings, supported version and version-group combination, and the branch identifier required at height h.
2. *Context-independent validity:* all applicable proof, signature, range and component-consistency checks.
3. *Context-dependent validity:* unspent and mature transparent inputs, eligible pool-specific anchors, unused nullifiers, applicable lock-time and expiry conditions, and upgrade-specific restrictions.
4. *Accounting:* the ordinary fee is nonnegative and all required individual and aggregate money ranges hold.

Acceptance produces a proposed state delta. No part of that delta becomes committed state if any required check fails.

The ordinary fee is determined by public quantities:

$$f(t) = v_{\text{in}}^{\text{transparent}}(t) - v_{\text{out}}^{\text{transparent}}(t) + \sum_{P \in \mathcal{P}} b_P(t) \geq 0. \tag{1}$$

An absent component contributes zero. A *JoinSplit* is an earlier shielded transfer component. Its formats have their own public transfer fields; at a compatibility boundary their net contribution must enter this equation with the specified sign. This formula is not permission to skip their own proofs or signatures.

Example 1.2 (The continuing payment)**.** An existing shielded note holds 80,000 zatoshis. The transaction pays 50,000, creates 20,000 of change and leaves 10,000 as a fee. There are no transparent inputs or outputs. Thus

$$b_P = 80{,}000 - 50{,}000 - 20{,}000 = 10{,}000 = f.$$

The pool loses 10,000 zatoshis of publicly accounted value. The payment and change amounts remain private. A transaction's fee is therefore public even when every payment component is shielded.

1.3 Rules, implementations and activation

A *network upgrade* changes the rules at a network-specific height. The rule-selection function $\mathsf{Rules}_\theta(h)$ determines allowed formats, branch identifiers, verification keys, pool restrictions and block-accounting parameters. The branch identifier is a domain-separation value; it is not a block hash or a replacement for the height test.

The normative baseline used here is `zips` commit `69610984`, notably the protocol specification sections "Transaction Encoding and Consensus", "Block Header Encoding and Consensus", "Proof of Work", and "Chain Value Pool Balances". The implementation cross-check is `zebra` commit `ef6325c0`, principally `zebra-consensus/src/block.rs`, `zebra-consensus/src/transaction.rs` and `zebra-state/src/service/check.rs`. References to later rule sets are conditional descriptions of their specified activation rules. The presence of code, a release number, or a ZIP marked Draft or Proposed does not establish activation on a network.

The compatibility boundary is deliberate. A complete node must also validate the active transparent and earlier shielded formats. This volume constructs the common ledger contract; it does not substitute that contract for their complete validation algorithms.

2 Blocks and branch-local validation

A transaction valid by itself may conflict with another transaction in the same block. A block valid on one branch may be invalid on another. These are not failures of a signature or proof. They are consequences of the state to which the transaction proposes to apply.

2.1 Header and body

Producing a block requires finding an *Equihash solution*, a string meeting prescribed collision conditions, and making the whole header's hash no larger than an integer *target*. These constitute its *proof of work*: a publicly checkable result of a computational search. The *nonce* is a header field varied during that search. A *transaction identifier* is its format's prescribed transaction digest. The first transaction, called the *coinbase*, is the special transaction permitted to claim new issuance and the block's fees. The modern header retains the following byte layout:

Field	Bytes
Signed version	4
Parent hash	32
Transaction Merkle root	32
Block commitments	32
Timestamp	4
Compact target, `nBits`	4
Nonce	32
Canonical encoded solution length	3
Equihash solution	1344

Fixed-width integer fields are little-endian. The length is a canonical *CompactSize* integer: values below 253 use one byte; otherwise prefixes `0xfd`, `0xfe` and `0xff` select two-, four- and eight-byte little-endian payloads, respectively, and a longer-than-necessary representation is rejected. Here the payload value is 1344, so its encoded length occupies three bytes.

The block hash is double SHA-256 of the complete header encoding, including the solution length and solution. Display conventions that reverse hash bytes must not leak into internal hash preimages. The transaction Merkle root commits to transaction identifiers in block order. Its binary tree duplicates the final child at a level having an odd number of children. This is not the zero-leaf padding rule of the authorising-data tree in §5.

The header is not the whole block. A header can satisfy proof of work while its body contains an invalid proof, a double spend, a wrong coinbase amount, or no available transactions. A node must obtain and validate the body before calling the block fully valid.

2.2 A validation algorithm

The following sequential algorithm defines a simple correctness model. An implementation may verify independent proofs concurrently, provided its final state and rejection conditions are equivalent.

1. Locate the candidate's parent and obtain that parent's validated state. Unknown ancestry is missing context, not evidence of validity.
2. Select the rules at the candidate height. Check header format, target, timestamp constraints and proof of work.
3. Parse the bounded body. Require at least one transaction, with coinbase first and no later coinbase. Check body commitments and all block-wide resource and encoding limits.
4. Evaluate ordinary transactions against a temporary view of the parent state. Consume transparent outputs and reserve nullifiers so that conflicts within the block are visible. Earlier transparent outputs in the block may become available subject to the applicable rules. Do not thereby make a newly computed note-tree root an eligible shielded anchor: modern shielded anchors refer to earlier blocks' final treestates.
5. Sum ordinary fees, validate the coinbase and prescribed allocations, and check all resulting chain pool balances. Append shielded commitments in transaction order and each component's output order. Check tree capacities and derive final treestates.
6. Construct the applicable history and authorising-data commitments. Compare every header commitment with its independently derived value.
7. Atomically store—make visible all together or not at all—the accepted block, its state delta and sufficient rollback information. Only then make this branch eligible for chain selection.

This algorithm names two different validation orderings. Ledger effects have a logical order fixed by the transaction sequence. CPU work such as signature checks can be reordered without changing that sequence. Batch verification therefore does not mean applying a transaction before all checks have completed.

A memory pool of unmined transactions is a node's local proposal set. It may reject transactions for fee, resource or expiry-risk policy beyond consensus validity. Acceptance into that set is neither inclusion in a block nor a promise that another node will relay the transaction.

2.3 Time and resource boundaries

The *median* here means: sort the $n > 0$ observations in increasing order and choose the element at zero-based index $\lfloor n/2 \rfloor$. For even n, this selects the upper of the two middle elements, not their average. This matters during the shorter initial windows.

In the pinned modern Mainnet rules, a header version must be at least 4, and a block is at most 2,000,000 bytes. A non-genesis timestamp must exceed the median of the preceding eleven timestamps, or all preceding timestamps if fewer exist. For the modern range it must also be at most that median plus 90 minutes. These are branch-computable rules. The extra rejection of a block more than two hours ahead of the node's own clock is time-dependent: the same block may become acceptable as the clock advances.

Transaction expiry and lock-time are independent of the header timestamp bounds. Their precise interpretation depends on format and rules. A wallet must not infer that a recent-looking timestamp makes an expired transaction valid, or that a transaction's expiry height determines the height at which it will be mined.

3 Work, difficulty, and chain selection

Proof of work gives a publicly checkable cost signal for a block. It does not prove its transactions valid, and it does not make the producer's identity or motives visible.

3.1 Equihash and the difficulty filter

Zcash uses Equihash with $(n, k) = (200, 9)$. A candidate solution selects $2^9 = 512$ distinct indices from 2^{21} possible indices. Each index selects a 200-bit string derived from the header prefix including its nonce. The selected strings must XOR to zero. Further conditions bind the solution to a binary collision tree: for every consecutive aligned group of 2^r indices, $1 \le r \le 8$, its XOR has $20r$ leading zero bits; for every level $1 \le r \le 9$, each left index sequence is lexicographically smaller than the corresponding right sequence.

For a zero-based index j, let $a = \lfloor j/2 \rfloor$ and $b = j \bmod 2$. Compute the 50-byte BLAKE2b digest of the 140-byte header prefix followed by the four-byte little-endian encoding of a, personalised by

$$\texttt{ZcashPoW} \,\|\, \mathrm{LE}_{32}(200) \,\|\, \mathrm{LE}_{32}(9).$$

The selected string is bytes $25b$ through $25b + 24$ of that digest. The 512 indices are encoded as consecutive 21-bit big-endian strings, giving $512 \cdot 21/8 = 1344$ bytes. Endianness here differs from the fixed integer header fields. These constructions instantiate the protocol specification's "Equihash" and "EquihashGen" sections; the `equihash` crate's verification algorithm is an implementation reference.

An Equihash solution must additionally pass the difficulty filter:

$$\mathrm{LEInt}(\mathsf{SHA256d}(\mathsf{header})) \le T.$$

The target T comes from the required `nBits`, not from a value freely chosen by the miner. Under the idealised uniform-hash model, one independent complete-header trial passes this inequality with probability $(T + 1)/2^{256}$. This model motivates the assigned work

$$w(B) = \left\lfloor \frac{2^{256}}{T(B) + 1} \right\rfloor. \tag{2}$$

The actual computational cost of generating Equihash candidates is not measured by this formula; work is the protocol's comparison unit.

3.2 Targets and compact representation

In the 32-bit integer `nBits`, bits 24–31 give the *exponent* e, bit 23 is the sign bit s, and bits 0–22 give the *mantissa* m. These names describe the scale and leading integer digits in the compact representation, not floating-point values. A nonnegative target decodes as $m\,256^{e-3}$, with an integer right shift when $e < 3$. A set sign bit decodes to zero under the specification's conversion; valid proof-of-work targets must satisfy the additional nonzero and limit rules. Target parsing is not floating-point arithmetic.

To encode a positive integer target x, first set

$$e = \left\lceil \frac{\text{bitlength}(x)}{8} \right\rceil, \qquad m = \left\lfloor x\, 256^{3-e} \right\rfloor.$$

If $m \geq 2^{23}$, divide m by 256 with truncation and increment e. Return $m + 2^{24}e$. Lower bits can be discarded, so encoding and then decoding generally rounds a target down. Consensus compares the required compact encoding; retaining extra precision locally must not create a different accepted target.

For modern Mainnet blocks well beyond the initial bootstrap range, let T_i and t_i be previous decoded targets and timestamps. The averaging window is 17 blocks; target spacing after Blossom is 75 seconds. Define

$$\begin{aligned} A &= 17 \cdot 75 = 1275, \\ M(h) &= \text{median}(t_{h-11}, \ldots, t_{h-1}), \\ E(h) &= M(h) - M(h-17), \\ D(h) &= A + \text{trunc}_0((E(h) - A)/4), \\ C(h) &= \max(1071, \min(1683, D(h))), \\ \overline{T}(h) &= \frac{1}{17} \sum_{i=h-17}^{h-1} T_i. \end{aligned}$$

Truncation trunc_0 is towards zero, not towards negative infinity. The bounds are $\lfloor 0.84A \rfloor = 1071$ and $\lfloor 1.32A \rfloor = 1683$. The next unencoded threshold is

$$T_{\text{next}} = \min\left(T_{\text{limit}}, \left\lfloor \frac{\overline{T}(h)}{A} \right\rfloor C(h)\right), \qquad T_{\text{limit}} = 2^{243} - 1.$$

Encoding this threshold produces the required `nBits`. Notice the division is rounded *before* multiplication by $C(h)$. Moving the floor can change consensus.

For example, if $\overline{T} = 1{,}275{,}000$, an observed interval of 1275 seconds leaves this illustrative unencoded threshold unchanged. Extreme short and long intervals saturate at 1,071,000 and 1,683,000. The numbers illustrate the arithmetic, not a network difficulty estimate. The executable check also exercises negative truncation and compact-target round trips.

At the chain's start, the specification uses available shorter medians, a proof-of-work-limit mean through the averaging-window bootstrap, and a special genesis threshold. Pre-Blossom blocks use their own spacing; Testnet has additional minimum-difficulty exceptions. An independent historical validator must use those branches rather than extrapolating the modern formula backwards.

3.3 Selecting a validated branch

For a chain $C = (B_0, \ldots, B_h)$, total work is $W(C) = \sum_i w(B_i)$. A node selects a fully valid visible branch with greatest total work. Height alone is not the comparison: many easier blocks can represent less work than fewer harder blocks. A branch does not become preferable merely because its header declares small targets; its blocks must actually satisfy the targets and the difficulty adjustment rules.

Equal-work ties need not produce instantaneous global agreement. Nodes can temporarily select different visible tips; block propagation and further work can resolve the tie. The rule says what a node does with its information. It does not promise that every node has identical information at every instant.

4 Supply and pool accounting

The proof system checks hidden transfers. Pool accounting checks the public boundary around those transfers. Neither replaces the other.

For a shielded pool initialised at zero, its branch balance is

$$B_P(C) = -\sum_{t \in C} b_P(t). \tag{3}$$

Thus an ordinary transaction updates B_P by $-b_P(t)$. A migration between pools P and Q, with amount v and fee f drawn from P, can have

$$b_P = v + f, \qquad b_Q = -v, \qquad \sum_R b_R = f.$$

The amounts crossing each pool boundary are public. The accounting does not identify which input notes financed them.

A required nonnegative pool balance prevents withdrawals beyond the pool's publicly recorded total. It does not prove that the remaining private notes are all valid. In particular, an invalid private creation could stay below that aggregate ceiling. Supply integrity still requires sound payment proofs, correct authorisation and uniqueness checks.

4.1 Coinbase and prescribed allocation

The first transaction is the *coinbase*. It claims the permitted block subsidy and ordinary transaction fees; it does not need ordinary spendable inputs to justify new issuance. It is subject to special format, destination and shielded-output rules. The ordinary fee equation must therefore not be applied to it as though it were an ordinary payment.

For the modern post-Blossom subsidy, let h_B be Blossom's activation height and define

$$H(h) = \left\lfloor \frac{h_B - 10{,}000}{840{,}000} + \frac{h - h_B}{1{,}680{,}000} \right\rfloor, \qquad s(h) = \left\lfloor \frac{1{,}250{,}000{,}000}{2 \cdot 2^{H(h)}} \right\rfloor.$$

This is the specification's time-spacing-adjusted halving formula after the slow-start and pre-Blossom intervals. It is a height function, not an estimate obtained from a calendar date.

A funding stream active over $[a, b)$ with prescribed numerator u and denominator v receives $\lfloor s(h)u/v \rfloor$ in that interval and zero outside it. The active rule set supplies the stream list and recipient rules. Amounts directed to the deferred pool are added to D; an authorised disbursement removes its prescribed amount from D and permits the corresponding coinbase allocation. Such a release is not new issuance. The miner's subsidy share is the block subsidy less required allocations; fees are additionally claimable as specified.

A validator must check both the aggregate amount claim and required payments. Knowing only the total subsidy is not enough to validate a coinbase. The full recipient schedules are compatibility data owned by the applicable funding ZIPs, not an alternative mathematical rule.

4.2 Maturity and spendability

Transparent coinbase outputs have a 100-block maturity rule. That rule should not be carelessly asserted for every kind of shielded coinbase output. The shielded protocols have their own coinbase construction and public-recovery requirements; the commitment format does not itself carry an ordinary transparent-output maturity flag. A wallet's chosen confirmation delay is another, separate condition. Distinguish node acceptance from the wallet's choice to risk a spend.

5 Authenticated block history

A header's parent hash links one block. A commitment to a structured history makes selected statements about earlier blocks efficiently checkable. The generic binary-tree and append-only-forest constructions are in the *Crypto Guide*, "Authenticated trees and append-only histories". This section fixes Zcash's concrete metadata, encodings and header commitments. No sampling protocol is needed to construct or validate them.

5.1 An epoch-scoped Merkle mountain range

For a candidate block B_n, let $x < n$ be the most recent network-upgrade activation height strictly below n. ZIP 221's history tree commits to

$$B_x, B_{x+1}, \ldots, B_{n-1}.$$

It excludes B_n itself. The delay avoids asking a block hash to commit to a structure containing that same hash.

The history is a forest of perfect binary trees, one per set bit of the leaf count. Its roots are called *peaks*. Append a new leaf as a height-zero peak; while the two rightmost peaks have equal height, replace them with their parent. Number stored nodes in creation order, placing a leaf before the parents completed by its append. The independent eleven-leaf example produces 19 stored nodes, with peak positions $14, 17, 18$ and heights $3, 1, 0$.

To obtain one root, combine peaks from left to right:

$$R = \mathsf{Parent}(\mathsf{Parent}(P_0, P_1), P_2)$$

for three peaks. These extra *bagging* combinations need not be stored as ordinary mountain nodes. A right fold would be a different root when there are at least three peaks. For one peak, the root node is that peak; its final root *digest* is still computed as below.

5.2 The complete node record

Every node is a canonical record, not just a hash. The initial record serialises the following fields in order:

1. 32-byte subtree commitment.
2. Earliest and latest timestamps, each unsigned 32-bit little-endian.
3. Earliest and latest compact targets, with the same integer encoding.
4. Earliest and latest final Sapling note-tree roots, each 32 bytes.
5. Total work, an unsigned 256-bit little-endian integer.
6. Earliest height, latest height and number of Sapling-bearing transactions, each canonical CompactSize.

The NU5 record appends earliest and latest final Orchard note-tree roots, then a CompactSize count of transactions with a nonempty Orchard component. The specified NU6.3 extension appends the analogous Ironwood root pair and transaction count. A count is of transactions, not Actions, notes or nonzero payments.

At a leaf, both endpoints of every pair come from that leaf's block. The subtree commitment is the ordinary block hash in internal byte order. Work is (2); each count is computed from the block body. For a parent, earliest fields come from the left child, latest fields from the right child, and work and counts are summed. Earliest means leftmost in chain order, not the numerically smallest timestamp: timestamps need not increase at every block.

For branch identifier β belonging to this history epoch, define

$$H_\beta(m) = \mathsf{BLAKE2b}_{256}(\texttt{ZcashHistory} \,\|\, \mathrm{LE}_{32}(\beta), m).$$

A parent's subtree commitment is

$$H_\beta(\mathsf{Serialize}(L) \,\|\, \mathsf{Serialize}(R)),$$

using the *entire* child records in the fixed order above. The final chain-history root is

$$H_\beta(\mathsf{Serialize}(R_{\text{root}})).$$

Hashing only child digest fields omits the authenticated metadata and produces the wrong tree.

The base record contains 144 fixed bytes and three CompactSize fields, so its length ranges from 147 to 171 bytes. The NU5 record contains 208 fixed bytes and four such fields, giving 212 to 244 bytes. The specified Ironwood extension gives 272 fixed bytes and five such fields, or 277 to 317 bytes. These are encoding bounds, not a claim that every extreme combination can occur in a real chain.

5.3 Upgrade boundaries and authenticated meaning

At the block activating the history-tree rule, Heartwood, the designated header field is the zero sentinel, not a root of an empty ordinary tree. At a later upgrade's activation block, the history commitment completes the preceding epoch. The next block commits to a single-leaf tree beginning with the activation block. The tree's hashing uses the branch identifier of the epoch represented by its nodes, including when the header carrying its completed root belongs to the next epoch. The history implementation and the upgrade selection together determine this boundary; applying a new personalisation to old-epoch nodes would change their committed root.

A valid inclusion path authenticates the supplied records against a chosen root under collision resistance. It does not, by itself, prove that every concealed subtree's declared work sum or count was honestly computed. Fully validating nodes recompute the metadata from accepted blocks. A client trusting only a root needs an additional reason to trust the relevant aggregate semantics. Hash binding is not arithmetic validation or proof of a best chain.

5.4 Authorising-data and block commitments

Transaction identifiers in version 5 commit to *effecting data*, the data specifying the transaction's effects. They exclude proofs and signatures. Each transaction therefore also has an authorising-data digest. ZIP 244 constructs that digest with branch-separated BLAKE2b from the format's transparent, Sapling and Orchard authorising-data component digests. The specified version 6 extension adds Ironwood and moves the affected anchors into authorising data. The payment-format verifier must compute the exact version's digest; an arbitrary hash of serialised transaction bytes is not a replacement.

Build the authorising-data tree in transaction order. A pre-v5 transaction has the prescribed leaf of 32 bytes of `0xff`. For versions defining an authorising digest, use that digest. Pad to the next power of two with 32-byte zero leaves. Since coinbase is mandatory, the leaf count is positive. Internal nodes are

$$H_{\text{auth}}(L, R) = \mathsf{BLAKE2b}_{256}(\texttt{ZcashAuthDatHash}, L\|R).$$

With one transaction there is no internal hash: the root is its leaf. For m transactions the tree height is $\lceil \log_2 m \rceil$.

From the ZIP 244 activation block, the header field is

$$\mathsf{hashBlockCommitments} = \mathsf{BLAKE2b}_{256}\big(\texttt{ZcashBlockCommit}, R_{\text{history}}\|R_{\text{auth}}\|0^{256}\big). \tag{4}$$

The final 32 bytes are an extension terminator. A later activated rule may replace that terminator with a further defined commitment structure; a full node must verify the complete active structure. It must not accept an arbitrary terminator because an older light client could treat it as an opaque extension.

There are now three distinct questions. A note-tree anchor can support a private input-existence proof. A transaction digest binds authorised transaction data. A block-history root authenticates selected earlier block records. Sharing a 32-byte representation does not make these objects interchangeable.

6 Reorganisation and confirmation confidence

A transaction's cryptographic validity is stable under unrelated extensions, but its inclusion and contextual validity are branch dependent. Chain selection can therefore change a wallet's apparent balance without breaking a commitment or forging a signature.

6.1 Rollback to a common ancestor

Suppose the selected branch changes from C to C', with last common ancestor a. Reconstruct S_a by undoing the removed suffix of C or loading its retained snapshot. Apply and validate C''s suffix in order. Undo data must cover transparent outputs, nullifier insertions, commitment appends, anchor availability, pool balances and history state. A note created only on the removed suffix is no longer an accepted input. A nullifier revealed only there is no longer spent on the new branch.

A transaction from the removed branch may be valid again, invalid because of a conflict, or expired. It is a candidate for re-evaluation, not an automatically accepted member of the new branch or memory pool. Likewise, a proof with an anchor existing only on the removed branch may need to be rebuilt. Persistence must not leave a mixture of old-branch nullifiers and new-branch roots.

Implementations can retain only a bounded rollback window or use checkpoints as safety and storage mechanisms. Such operational bounds are not a theorem that a deeper competing chain is cryptographically impossible. The protocol's most-work criterion and a program's recovery policy are different statements.

6.2 An explicit probabilistic model

Consider an idealised two-producer model with $0 < p < 1$ and a positive integer confirmation count z. Independent block arrivals belong to an honest branch with probability p and an adversarial branch with probability $q = 1 - p$. Both branches have equal per-block work; network delays, changing hash rates and difficulty feedback are ignored. The attacker begins from the same fork point as the payment. Ties count as attacker success. These assumptions define the calculation; they are not observations guaranteed by a real network. The separate endpoint $p = 1, q = 0$ has no adversarial arrivals and reversal probability zero.

For a current honest lead $d > 0$, let u_d be the probability that the attacker ever reaches a tie. First-step conditioning gives

$$u_d = p\,u_{d+1} + q\,u_{d-1}, \qquad u_0 = 1.$$

When $q < p$, solve first on the finite interval $0, \dots, M$, with failure at M. Substitution verifies

$$u_d^{(M)} = \frac{(q/p)^d - (q/p)^M}{1 - (q/p)^M}.$$

As M increases, the events of reaching zero before M increase to the event of ever reaching zero: any finite successful path has a finite maximum. Taking the limit proves $u_d = (q/p)^d$. For $q = p$, the finite-interval solution is $1 - d/M$ and the limit is one; for $q > p$, the same finite-interval formula tends to one. A strict overtake convention instead requires a different boundary; silently switching between tie and overtake changes the answer.

Waiting for z honest blocks does not give the attacker zero blocks. Let K be their block count when the zth honest block arrives. For $k \geq 0$,

$$\Pr[K = k] = \binom{z + k - 1}{k} p^z q^k.$$

Indeed, the last arrival is honest, while the preceding $z + k - 1$ arrivals contain $z - 1$ honest arrivals and k adversarial arrivals. Their possible orders give the binomial coefficient.

Conditioning on K and using the catch-up probability gives, for $q < p$,

$$P_z = 1 - \sum_{k=0}^{z-1} \binom{z+k-1}{k} p^z q^k \bigl(1 - (q/p)^{z-k}\bigr). \tag{5}$$

The finite sum combines the $K < z$ cases; $K \geq z$ already reaches a tie. At $q = 0.1$ and $z = 6$, it evaluates exactly to 0.00059141216, approximately 0.059141216%. At $z = 1$ it gives 0.2, not q/p: the latter assumes a fixed known deficit and ignores the attacker's work during the wait. The draft's numerical check uses exact rational arithmetic.

6.3 What a confirmation means

A confirmation policy selects an acceptable risk under assumptions about the adversary and network. It cannot turn (5) into an unconditional finality guarantee. The model also says nothing about a client whose peers control and restrict its entire network view (an *eclipse*), or about a transaction that fails its payment proof.

The complete chain of reasoning is therefore conditional and layered: validated transactions produce valid branch-local state; valid blocks supply admissible work; the most-work rule selects among visible valid branches; a stated network and adversary model supports a probability of later reversal. Each layer answers a different question.

Evidence and executable checks. The concrete history rules are ZIP 221, ZIP 244 and the conditional Ironwood extension in ZIP 258; implementation checks use `zcash_history` and `zebra-chain/src/history_tree.rs`. The public specifications are available at ZIP 221, ZIP 244 and ZIP 258. The companion `check_deployed.py` checks payment signs, compact integers, target rounding, difficulty examples, the confirmation sum, history positions and record-size arithmetic. These checks do not prove hash hardness, honest-majority assumptions, or the correctness of every implementation of the consensus rules.